

ACTIVITAT D'AVALUACIÓ CONTÍNUA UF24040EC02.

Nom i Cognoms de l'alumne/a	
NIF de l'alumne/a	
Data de la prova	
Signatura	

Número i nom del curs	25/FOAP/781/0195046/001 IFCD0112 Programació amb llenguatges orientats a objectes i bases de dades relacionals
Mòdul Professional que s'avalua	MF0227_3: Programació orientada a objectes UF2404: Principis de la programació orientada a objectes
ACTIVITAT 1	10 preguntes practics
DESCRIPCIÓ	10 preguntes d'avaluació de la UF2404: Principis de la programació orientada a objectes (Python)

Graella d'avaluació		NOTA
Execució i/o Resultat	1	
	2	
	3	
	4	
	5	
	6	
	7	
	8	
	9	
	10	
Puntuació		

Signatura del formador/a



Generalitat de Catalunya



Examen tècnic / Examen técnico.

Programació Orientada a Objectes en Python

Recursos permesos / Recursos permitidos: apunts, Internet, documentació i eines d'IA.

Total: 10 exercicis.

Normes / Normas

CAT: Es pot utilitzar qualsevol recurs extern, però l'alumne ha de ser capaç d'explicar i modificar el seu codi. El professor pot demanar un petit canvi en qualsevol exercici abans de donar-lo per finalitzat.

ES: Se puede utilizar cualquier recurso externo, pero el alumno debe ser capaz de explicar y modificar su código. El profesor podrá pedir un pequeño cambio en cualquiera de los ejercicios antes de considerarlo finalizado.

No es valorarà només que el programa funcioni / No se valorará únicamente que el programa funcione.

També es valorarà / También se valorará:

- disseny de les classes / diseño de las clases;
- responsabilitat de cada objecte / responsabilidad de cada objeto;
- encapsulació / encapsulación;
- gestió d'errors / gestión de errores;
- qualitat i llegibilitat del codi / calidad y legibilidad del código;
- capacitat de justificar les decisions / capacidad para justificar las decisiones.



Generalitat
de Catalunya



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

Exercici 1 — Reparar un disseny defectuós

Ejercicio 1 — Reparar un diseño defectuoso

Es proporciona el següent codi / Se proporciona el siguiente código:

```
class Usuario:
    total = 0

    def __init__(self, nombre, cursos=[]):
        self.nombre = nombre
        self.cursos = cursos
        total += 1

    def agregar_curso(self, curso):
        cursos.append(curso)

u1 = Usuario("Ana")
u2 = Usuario("Marc")

u1.agregar_curso("Python")

print(u1.cursos)
print(u2.cursos)
print(Usuario.total)
```

CAT:

El programa conté diversos errors, alguns de sintaxi o execució i altres relacionats amb el disseny orientat a objectes.

Has de:

1. identificar almenys **4 problemes**;
2. corregir-los;
3. explicar breument per què cada canvi és necessari;
4. aconseguir que cada usuari tingui la seva pròpia llista de cursos;
5. aconseguir que `Usuario.total` indiqui correctament quants usuaris s'han creat.

ES:

El programa contiene varios errores, algunos de ejecución y otros relacionados con el diseño orientado a objetos.

Debes:

1. identificar al menos **4 problemas**;
2. corregirlos;
3. explicar brevemente por qué es necesario cada cambio;
4. conseguir que cada usuario tenga su propia lista de cursos;
5. conseguir que `Usuario.total` indique correctamente cuántos usuarios se han creado.



Exercici 2 — Reserva de places

Ejercicio 2 — Reserva de plazas

CAT:

Crea una classe **Esdeveniment** que representi un esdeveniment amb un nombre màxim de places.

Ha de permetre:

- reservar(persona)
- cancelar(persona)
- places_disponibles()

Condicions:

- una persona no pot reservar dues vegades;
- no es poden superar les places disponibles;
- no es pot cancel·lar una reserva que no existeix;
- la llista interna de persones inscrites no s'ha de poder modificar directament des de fora.

Afegeix també:

- len(esdeveniment)

que ha de retornar el nombre de persones inscrites.

ES:

Crea una classe **Evento** que represente un evento con un número máximo de plazas.

Debe permitir:

- reservar(persona)
- cancelar(persona)
- plazas_disponibles()

Condiciones:

- una persona no puede reservar dos veces;
- no pueden superarse las plazas disponibles;
- no puede cancelarse una reserva inexistente;
- la lista interna de personas inscritas no debe poder modificarse directamente desde fuera.

Además:

- len(evento)

debe devolver el número de personas inscritas.



Exercici 3 — Figures sense modificar la funció

Ejercicio 3 — Figuras sin modificar la función

Es proporciona / Se proporciona:

```
def imprimir_informe(figuras):  
    for figura in figuras:  
        print(  
            figura.nombre(),  
            round(figura.area(), 2),  
            round(figura.perimetro(), 2)  
        )
```

CAT:

Crea les classes necessàries perquè la funció anterior funcioni amb:

- `Rectangle`;
- `Cercle`;
- `TriangleRectangle`.

No pots modificar `imprimir_informe()`.

Cada figura ha d'implementar:

- `nombre()`
- `area()`
- `perimetro()`

Després afegeix una nova figura `Quadrat` fent el mínim de codi duplicat possible.

ES:

Crea las clases necesarias para que la función anterior funcione con:

- `Rectangulo`;
- `Circulo`;
- `TrianguloRectangulo`.

No puedes modificar `imprimir_informe()`.

Cada figura debe implementar:

- `nombre()`
- `area()`
- `perimetro()`

Después añade una nueva figura `Cuadrado` intentando duplicar la menor cantidad posible de código.



Exercici 4 — Historial bancari

Ejercicio 4 — Historial bancario

CAT:

Crea una classe **CompteBancari**.

Ha de permetre:

- ingressar(quantitat)
- retirar(quantitat)
- transferir(compte_desti, quantitat)

Cada operació ha de quedar guardada en un historial.

Exemple conceptual:

- INGRES +100
- RETIRADA -30
- TRANSFERENCIA -20

Condicions:

- no es permeten quantitats negatives o zero;
- no es pot retirar més diners dels disponibles;
- una transferència ha de modificar correctament els dos comptes;
- l'historial retornat a l'usuari no ha de permetre modificar l'historial intern real.

Implementa:

- obtenir_historial()

ES:

Crea una clase **CuentaBancaria**.

Debe permitir:

- ingresar(cantidad)
- retirar(cantidad)
- transferir(cuenta_destino, cantidad)

Cada operación debe quedar almacenada en un historial.

Condiciones:

- no se permiten cantidades negativas o cero;
- no se puede retirar más dinero del disponible;
- una transferencia debe modificar correctamente ambas cuentas;
- el historial devuelto al usuario no debe permitir modificar el historial interno real.

Implementa:

- obtener_historial()



Exercici 5 — Refactorització

Ejercicio 5 — Refactorización

Es proporciona / Se proporciona:

```
class Pedido:
    def calcular_precio(self, tipo, precio, cantidad):
        if tipo == "normal":
            return precio * cantidad
        elif tipo == "vip":
            return precio * cantidad * 0.8
        elif tipo == "empleado":
            return precio * cantidad * 0.5
        elif tipo == "premium":
            return precio * cantidad * 0.7
```

-

CAT:

Aquest disseny obliga a modificar `Pedido` cada vegada que apareix un nou tipus de client.

Refactoritza el programa utilitzant POO perquè sigui possible crear nous tipus de descompte sense modificar la classe que calcula la comanda.

Implementa almenys:

- client normal;
- client VIP;
- empleat;
- client premium.

Després afegeix un nou tipus:

- ClientEstudiant → 15% de descompte

sense modificar la lògica principal de `Pedido`.

ES:

Este diseño obliga a modificar `Pedido` cada vez que aparece un nuevo tipo de cliente.

Refactoriza el programa utilizando POO para que sea posible crear nuevos tipos de descuento sin modificar la clase que calcula el pedido.

Implementa al menos:

- cliente normal;
- cliente VIP;
- empleado;
- cliente premium.

Después añade:

- ClienteEstudiante → 15% de descuento

sin modificar la lógica principal de `Pedido`.



Generalitat
de Catalunya



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

Exercici 6 — Sistema d'inventari

Ejercicio 6 — Sistema de inventario

CAT: Crea les classes necessàries per gestionar un inventari.

Cada **Producte** té:

- codi
- nom
- preu
- stock

L'**Inventari** ha de permetre:

- `afegir_producte(producte)`
- `eliminar_producte(codi)`
- `buscar(codi)`
- `vendre(codi, quantitat)`
- `reposar(codi, quantitat)`
- `valor_total()`

Condicions:

- dos productes no poden tenir el mateix codi;
- no es pot vendre més stock del disponible;
- preu, quantitat i stock han de ser vàlids;
- `valor_total()` retorna el valor monetari de tot l'estoc;
- la col·lecció interna de productes no ha de quedar exposada directament.

ES: Crea las clases necesarias para gestionar un inventario.

Cada **Producto** tiene:

- `codigo`
- `nombre`
- `precio`
- `stock`

El **Inventario** debe permitir:

- `agregar_producto(producto)`
- `eliminar_producto(codigo)`
- `buscar(codigo)`
- `vender(codigo, cantidad)`
- `reponer(codigo, cantidad)`
- `valor_total()`

Condiciones:

- dos productos no pueden tener el mismo código;
- no puede venderse más stock del disponible;
- precio, cantidad y stock deben ser válidos;
- `valor_total()` devuelve el valor monetario de todo el stock;
- la colección interna de productos no debe qu te.



Generalitat
de Catalunya



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

Exercici 7 — Classes que han de funcionar juntes

Ejercicio 7 — Clases que deben funcionar juntas

CAT: Implementa un sistema de cursos.

Crea:

- Persona
- Alumne
- Professor
- Curs

Alumne i **Professor** han d'heretar de **Persona**.

Un **Curs** té:

- un nom;
- un professor;
- diversos alumnes;
- una capacitat màxima.

Ha de permetre:

- matricular(alumne)
- desmatricular(alumne)
- canviar_professor(professor)

Condicions:

- només es poden matricular objectes **Alumne**;
- només un **Professor** pot ser professor del curs;
- un alumne no es pot matricular dues vegades;
- s'ha de respectar la capacitat màxima.

Implementa també `__str__()` perquè un curs es pugui imprimir de manera llegible.



ES Implementa un sistema de cursos.

Crea:

- Persona
- Alumno
- Profesor
- Curso

Alumno y **Profesor** deben heredar de **Persona**.

Un **Curso** tiene:

- un nombre;
- un profesor;
- varios alumnos;
- una capacidad máxima.

Debe permitir:

- matricular(alumno)
- desmatricular(alumno)
- cambiar_profesor(profesor)

Condiciones:

- únicamente pueden matricularse objetos **Alumno**;
- solamente un **Profesor** puede ser profesor del curso;
- un alumno no puede matricularse dos veces;
- debe respetarse la capacidad máxima.

Implementa también `__str__()` para mostrar un curso de forma legible.

Exercici 8 — Codi desconegut



Generalitat
de Catalunya



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

Ejercicio 8 — Código desconocido

Sense executar primer el programa / Sin ejecutar primero el programa:

```
class A:
    def metodo(self):
        return "A"

class B(A):
    def metodo(self):
        return "B" + super().metodo()

class C(A):
    def metodo(self):
        return "C" + super().metodo()

class D(B, C):
    def metodo(self):
        return "D" + super().metodo()

obj = D()
print(obj.metodo())
print(D.mro())
```

CAT:

1. Escribe quin resultat creus que mostrarà.
2. Explica per què.
3. Executa'l i comprova la resposta.
4. Modifica únicament l'ordre de l'herència de **D** i explica com canvia el resultat.
5. Explica què està fent `super()` realment en aquest exemple.

ES:

1. Escribe qué resultado crees que mostrará.
2. Explica por qué.
3. Ejecútalo y comprueba la respuesta.
4. Modifica únicamente el orden de herencia de **D** y explica cómo cambia el resultado.
5. Explica qué está haciendo realmente `super()` en este ejemplo.



Exercici 9 — Sistema de préstec

Ejercicio 9 — Sistema de préstamos

CAT: Crea un sistema de biblioteca amb:

- Llibre
- Usuari
- Prestec
- Biblioteca

Un llibre pot estar disponible o prestat.

La biblioteca ha de permetre:

- `afegir_llibre(llibre)`
- `registrar_usuari(usuari)`
- `prestar(isbn, id_usuari)`
- `retornar(isbn)`
- `prestecs_actius()`

Condicions:

- cada llibre ha de tenir un ISBN únic;
- cada usuari ha de tenir un identificador únic;
- no es pot prestar un llibre que ja està prestat;
- cada usuari pot tenir com a màxim 3 llibres simultàniament;
- un **Prestec** ha de relacionar un llibre i un usuari;
- retornar un llibre ha de finalitzar el préstec corresponent.

No guardis simplement noms o ISBN dins de **Prestec**: fes servir objectes.

ES: Crea un sistema de biblioteca con:

- Libro
- Usuario
- Prestamo
- Biblioteca

Un libro puede estar disponible o prestado. La biblioteca debe permitir:

- `agregar_libro(libro)`
- `registrar_usuario(usuario)`
- `prestar(isbn, id_usuario)`
- `devolver(isbn)`
- `prestamos_activos()`

Condiciones:

- cada libro debe tener un ISBN único;
- cada usuario debe tener un identificador único;
- no puede prestarse un libro que ya esté prestado;
- cada usuario puede tener como máximo 3 libros simultáneamente;
- un **Prestamo** debe relacionar un libro y un usuario;
- devolver un libro debe finalizar el préstamo correspondiente.

No almacenes simplemente nombres o ISBN dentro de **Prestamo**: utiliza objetos.



Exercici 10 — Sistema extensible de comandes

Ejercicio 10 — Sistema extensible de pedidos

Has de construir un petit sistema complet. / Debes construir un pequeño sistema completo.

CAT:

Una botiga gestiona comandes amb diferents productes.

Existeixen tres tipus inicials:

- ProducteFísic
- ProducteDigital
- Subscripció

Tots comparteixen:

- id
- nom
- preu_base

Però el preu final es calcula diferent.

Producte físic

Preu final:

- $\text{preu_base} + \text{cost_enviament}$

Producte digital

Preu final:

- preu_base

Subscripció

Rep també el nombre de mesos:

- $\text{preu_base} * \text{mesos}$

Crea també una classe **Comanda**.

Ha de permetre:

- `afegir(producte)`
- `eliminar(id_producte)`
- `calcular_total()`



Ara afegeix les següents regles:

- una comanda no pot tenir dos productes amb el mateix `id`;
- tots els preus han de ser vàlids;
- `calcular_total()` no ha de preguntar quin tipus de producte està processant amb `if type(...)`;
- la comanda ha de funcionar sense modificacions si després apareix una nova classe de producte compatible.

Finalment crea:

- `ProducteDescompte`

que rep un percentatge de descompte.

Integra'l a la comanda **sense modificar** `calcular_total()`.

Segona part

El professor indicarà durant l'examen **un canvi no anunciat** que hauràs d'incorporar al programa sense redissenyar-lo completament.

Exemples de canvi possibles:

- - IVA diferent segons producte.
- - Quantitat de productes.
- - Cost d'enviament gratuït a partir d'un import.
- - Nova classe de producte.
- - Aplicació d'un cupó.

ES:

Una tienda gestiona pedidos con distintos tipos de productos.

Existen inicialmente:

- `ProductoFisico`
- `ProductoDigital`
- `Suscripcion`

Todos comparten:

- `id`
- `nombre`
- `precio_base`

pero calculan su precio final de forma distinta.

Producto físico

- `precio_base + coste_envio`

Producto digital

- `precio_base`



Suscripción

Recibe además el número de meses:

- `precio_base * meses`

Crea también una clase `Pedido`.

Debe permitir:

- `agregar(producto)`
- `eliminar(id_producto)`
- `calcular_total()`

Añade estas condiciones:

- un pedido no puede contener dos productos con el mismo `id`;
- todos los precios deben ser válidos;
- `calcular_total()` no debe preguntar qué clase de producto está procesando mediante `if type(...)`;
- el pedido debe seguir funcionando sin modificaciones si aparece un nuevo tipo de producto compatible.

Finalmente crea:

- `ProductoDescuento`

que recibe un porcentaje de descuento.

Intégralo en el pedido **sin modificar** `calcular_total()`.

Segunda parte

El profesor indicará durante el examen **un cambio no anunciado** que el alumno deberá incorporar sin rediseñar completamente la aplicación.

Posibles ejemplos:

- - IVA diferente según el producto.
 - - Cantidades.
 - - Envío gratuito a partir de determinado importe.
 - - Nuevo tipo de producto.
 - - Aplicación de un cupón.
-

Defensa obligatòria / Defensa obligatoria

Després de completar els exercicis, el professor seleccionarà almenys **dos exercicis**.

L'alumne haurà de:

1. explicar què fa cada classe;
2. explicar per què ha utilitzat herència, composició o encapsulació;
3. indicar un possible error o cas límit;
4. modificar una petita part del programa en directe;
5. predir què passarà abans d'executar el canvi.

No serà suficient que el codi funcioni si l'alumne no pot explicar-lo.

Criteris d'avaluació / Criterios de evaluación

40% — Funcionament / Funcionamiento

El programa compleix els requisits i gestiona correctament els casos límit.

25% — Disseny POO / Diseño POO

Responsabilitats de classes, encapsulació, herència, polimorfisme i composició.

15% — Qualitat del codi / Calidad del código

Claredat, noms, absència de duplicació innecessària i estructura.

20% — Defensa i modificació / Defensa y modificación

Capacitat d'explicar, predir i modificar el programa.



Sistema de selecció i puntuació / Sistema de selección y puntuación

CAT — Català

En aquest examen heu de seleccionar i resoldre **exactament 5 dels 10 exercicis proposats**.

No tots els exercicis tenen el mateix valor. Els exercicis amb més dificultat tenen una puntuació més alta.

Exercici	Puntuació màxima
1	0,5 punts
2	0,6 punts
3	0,8 punts
4	0,8 punts
5	1,0 punt
6	0,8 punts
7	1,2 punts
8	2,1 punts
9	2,7 punts
10	3,3 punts

Com funciona la nota?

La nota final serà la suma dels punts obtinguts en els **5 exercicis seleccionats**.

Per aprovar cal obtenir un mínim de:

5 punts sobre 10

La nota màxima final serà **10**, encara que la combinació d'exercicis seleccionada permeti sumar més de 10 punts.

Important

No és possible aprovar seleccionant únicament els exercicis més fàcils.

Per exemple, si seleccioneu els exercicis:

1 + 2 + 3 + 4 + 6

la puntuació màxima possible seria:



**Generalitat
de Catalunya**



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

$0,5 + 0,6 + 0,8 + 0,8 + 0,8 = 3,5$ punts

Per tant, encara que aquests cinc exercicis estiguessin perfectament resolts, la nota seria **3,5** i no es podria aprovar.

Haureu d'escollir bé la vostra estratègia i assumir com a mínim algun exercici de dificultat elevada.

Exercicis 8, 9 i 10

Els exercicis **8, 9 i 10** tenen una ponderació superior perquè requereixen una comprensió més profunda de la programació orientada a objectes.

Per aprovar l'examen, haureu de seleccionar **com a mínim un dels exercicis 8, 9 o 10**.

Ús d'apunts, Internet i IA

Podeu utilitzar:

- apunts;
- documentació;
- Internet;
- eines d'intel·ligència artificial.

Tot i això, utilitzar aquestes eines no garanteix la puntuació de l'exercici.

Haureu de ser capaços de:

- entendre el codi que entregueu;
- explicar les classes i els mètodes;
- justificar les decisions de disseny;
- detectar errors;
- modificar el programa si es demana;
- respondre preguntes sobre la vostra solució.

El professor podrà demanar una petita modificació o explicació d'algun dels exercicis seleccionats.

Un exercici que funcioni però que no pugueu explicar o modificar podrà no obtenir la puntuació completa.



ES — Español

En este examen debéis seleccionar y resolver **exactamente 5 de los 10 ejercicios propuestos**.

No todos los ejercicios tienen el mismo valor. Los ejercicios de mayor dificultad tienen una puntuación más alta.

Ejercicio	Puntuación máxima
1	0,5 puntos
2	0,6 puntos
3	0,8 puntos
4	0,8 puntos
5	1,0 punto
6	0,8 puntos
7	1,2 puntos
8	2,1 puntos
9	2,7 puntos
10	3,3 puntos

¿Cómo funciona la nota?

La nota final será la suma de los puntos obtenidos en los **5 ejercicios seleccionados**.

Para aprobar es necesario obtener como mínimo:

5 puntos sobre 10

La nota máxima final será **10**, aunque la combinación de ejercicios seleccionada permita sumar más de 10 puntos.

Importante

No es posible aprobar seleccionando únicamente los ejercicios más fáciles.

Por ejemplo, si seleccionáis los ejercicios:

$$1 + 2 + 3 + 4 + 6$$

la puntuación máxima posible sería:

$$0,5 + 0,6 + 0,8 + 0,8 + 0,8 = 3,5 \text{ puntos}$$

Por tanto, aunque los cinco ejercicios estuvieran perfectamente resueltos, la nota sería **3,5** y no se podría aprobar.

Tendréis que elegir bien vuestra estrategia y asumir al menos algún ejercicio de dificultad elevada.



Generalitat
de Catalunya



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

Ejercicios 8, 9 y 10

Los ejercicios **8, 9 y 10** tienen una ponderación superior porque requieren una comprensión más profunda de la programación orientada a objetos.

Para aprobar el examen, deberéis seleccionar **como mínimo uno de los ejercicios 8, 9 o 10**.

Uso de apuntes, Internet e IA

Podéis utilizar:

- apuntes;
- documentación;
- Internet;
- herramientas de inteligencia artificial.

Sin embargo, utilizar estas herramientas no garantiza la puntuación del ejercicio.

Debéis ser capaces de:

- entender el código que entregáis;
- explicar las clases y los métodos;
- justificar las decisiones de diseño;
- detectar errores;
- modificar el programa si se solicita;
- responder preguntas sobre vuestra solución.

El profesor podrá solicitar una pequeña modificación o explicación de alguno de los ejercicios seleccionados.

Un ejercicio que funcione pero que no podáis explicar o modificar podrá no obtener la puntuación completa.